SOVEREIGN SHARDS — TECHNICAL MIGRATION LOG

Project: sovereign-shards

Branch: `main`

System Clock Baseline: June 2026

Logger Architecture: Eidetic Memory (Living, Context, Working Matrices)

Historical Line Count: ~2,552 lines

TL;DR (Executive Summary)

This log represents the definitive history of the Sovereign Shards framework—an air-gapped, zero-dependency, local multi-agent system built to run on standard 16GB consumer-grade hardware within a strict **4.5 GB collective memory budget**. This log tracks the transition away from monolithic, high-overhead cloud models to an on-device "Tiny Brains" orchestration matrix (sub-3B models) backed by a 0ms-latency **Zero-Inference Router** and a fault-tolerant multi-turn continuation mechanism (`.j_chain.json`).

1. REPOSITORY MILESTONES (M1 – M17)

M1: Dynamic Tool Registry

- **Scope:** Introduced `app.agent` package and `ToolRegistry` to unify built-in and script-backed tools.
- **Key Files:** `app/agent/__init__.py` , `app/agent/tool_registry.py`
- **Actions:** Implemented `ToolSpec` metadata models, the `ScriptTool` subprocess wrapper, and dynamic discovery of files in `tools/run/` as `run_<script_name>` .

M2: Chat-Loop Integration

- **Scope:** Replaced legacy ad-hoc tool dispatch in `app/chat.py` with registry-driven execution and model prompt blocks.
- **Key Files:** `app/chat.py`
- **Actions:** Injected `ToolRegistry` into the `run_chat` startup pathway, aligned system prompts with structured `ACTION` templates, and implemented the strict **3-hop tool-call limit per turn**.

M3: Quick-Build Intent

- **Scope:** Added fast-path interceptors to bypass model inference when direct scaffolding is requested.
- **Key Files:** `app/chat.py`

`run_scaffold` via the registry at 0ms LLM latency.

M4: Hardened File Tools & FAT32 Limits

- **Scope:** Upgraded storage helper files to enforce compliance with consumer-media platforms.
- **Key Files:** `app/file_tools.py`
- **Actions:** Enforced `MAX_FILE_BYTES = 4GB` to respect FAT32 USB formatting boundaries. Implemented chunked reads with precise offset markers.

M5: Script-Tool Metadata Manifest

- **Scope:** Introduced `tools/run/registry.json` schema files.
- **Key Files:** `tools/run/registry.json`, `app/agent/tool_registry.py`
- **Actions:** Created static schema checks for all discoverable script arguments, preventing runtime model parsing crashes.

M6: Manual Operations Doc

- **Scope:** Added explicit user commands and runtime configurations to allow seamless manual debugging.
- **Key Files:** `run.py`, `docs/USER_MANUAL.md`

M7: Deterministic Sovereign Tool Layer Rebuild

- **Scope:** Replaced loose execution boundaries with a strict, dictionary-compatible, schema-aware, and side-effect-aware router pipeline.
- **Key Files:** `app/agent/tool_registry.py`, `app/agent/tool_schema.py`, `app/agent/script_tool.py`, `app/agent/tool_router.py`
- **Actions:** Implemented dict-like accessor methods on the registry, validated parameter types against strict schemas before execution, and initialized the `exec` and `network` policy enforcement restrictions.

M8: Live Hardware Validation & Local Hardening

- **Scope:** Resolved 17 critical bugs during live USB-hardware execution on FAT32 media.
- **Key Files:** `app/chat.py`, `tools/run/bash.py`, `tools/run/read.py`, `tools/run/search.py`
- **Actions:** Replaced Windows thread-blocking `Popen` structures in `bash.py` with stable subprocess runs, introduced automatic argument-swapping for `run_search` via `os.path.isfile` heuristics, and forced UTF-8 encoding outputs to prevent Windows `cp1252` encoding crashes.

M9: Endurance Testing & Router Maturation

- **Scope:** Ran three rounds of rigorous 20-turn endurance testing to expand router capabilities.
- **Key Files:** `app/router.py`, `prompts/J-system.txt`

default pointer for `list_dir`, and appended the identity-lock and anti-Chinese language prompts to system instructions.

M10: Personality Layer Integration

- **Scope:** Codified J's calm, precise, and sardonic persona directly into the framework layer at zero inference cost.
- **Key Files:** `app/personality.py`, `app/ui.py`
- **Actions:** Created 35+ pre-written response functions with randomized variations to replace raw system terminal print strings.

M11: High-Budget Multi-Step Pipelines

- **Scope:** Handled high-budget execution pipelines (up to 25 calls) without looping or context exhaustion.
- **Key Files:** `app/chat.py`, `app/agent/circuit_breaker.py`
- **Actions:** Scaled the circuit breaker ceiling dynamically using `max_step_turns = max(12, tool_budget + 10)`, implemented a pre-execution deduplication cache guard, and introduced regex rescue patterns to parse unescaped quotes inside JSON strings.

M12: Continuity Anchors for Serial Reads

- **Scope:** Stopped target drift during multi-file inspection sequences.
- **Key Files:** `app/chat.py`
- **Actions:** Automated file enumeration when processing target directory scans and appended unresolved checklist items directly into the phase compression summary.

M13: Context Persistence & Reflection Batching

- **Scope:** Stopped memory loss and context degradation during long-horizon runs.
- **Key Files:** `app/agent/context.py`, `app/agent/reflection.py`
- **Actions:** Implemented automated scratch pad fact extraction to disk, set up task accumulators, and implemented batched compression algorithms to truncate memory arrays before LLM consumption.

M14: Chain Checkpoint/Resume System

- **Scope:** Introduced the primary continuation mechanism for multi-turn execution.
- **Key Files:** `app/chat.py`, `.gitignore`
- **Actions:** Built automatic state checkpoint serializations (`.j_chain.json`) that trigger when the local turn budget is exhausted. This allows J to pause execution, return a summary to the user, and resume contextually on the next command prompt.

M15: CI/CD Pipeline & Ruff Linting

- **Scope:** Codified static analysis gates to ensure code quality and regression safety.

- **Actions:** Wired up multi-version Python verification tests (3.10 through 3.12) and established strict Ruff lint rules.

M16: J Cloud Platform

- **Scope:** Built a browser-accessible, Convex-backed deployment of the sovereign engine.
- **Key Files:** `src/pages/ChatPage.tsx` , `convex/tools.ts` , `convex/llm.ts`
- **Actions:** Designed a dark-themed visual layout, integrated GitHub Git Data APIs to enable atomic multi-file pushes, and configured model failover mechanics.

M17: Self-Extending Tool Forge

- **Scope:** Enabled J to dynamically write and register its own tools.
- **Key Files:** `convex/tools.ts`
- **Actions:** Implemented the tool builder engine to compile, validate, and write new Python modules to the code tree.

2. CHRONOLOGICAL SESSION RECORDS (S34 – S35)

Session 34 — CI Recovery & Security Hardening

- **Date:** May 24, 2026
- **Contributor:** External Contributor Pass (@Gh0stlyKn1ght)
- **Vulnerabilities Resolved:** Malformed nested triple-quoted f-strings inside `tool_forge.py` were crashing the main compiler. Broad Ruff suppresses were hiding real warnings.
- **Hardening Measures:**
  1. Pinned GitHub actions to exact commit SHAs and added Bandit security scanners to the pre-push pipeline.
  2. Implemented strict command denylists in `run_bash` to block destructive shell commands (`rm -rf /` , netcat patterns).
  3. Forced the GitHub-agent registry into dry-run/read-only mode when running in unprivileged CI environments.
  4. Introduced path-traversal validation checks to block directory escape attempts.

Session 35 — Boot Failure Resolution & Repo Hygiene

- **Date:** May 29, 2026
- **Agent:** Claude Opus 4.8 (via Claude Code)
- **Root Cause Diagnosis:** Commit `8347928` replaced the functional 1393-line monolith inside `app/chat.py` with a 270-line abstract skeleton. This skeleton imported non-existent modules (`JarvisOneForAll` , `HamiltonExecutor` , `SovereignPlanner`), making the entire application dead-on-arrival (throwing `ModuleNotFoundError` on boot).
- **Remediation:**

(`f340b39`), returning J instantly to active service.

2. Staged and committed 254 project-reorganization file moves as clean, traceable renames to preserve history.
3. Hardened `_path_guard.py` and `app/file_tools.py` to validate characters and drop inputs containing carriage returns or strings longer than 255 characters (preventing the model from passing entire task instructions as directory names).
4. Added `model-server/` , `python/` , and `get-pip.py` to `.gitignore` to prevent committing massive local platform binaries.

APPENDIX A: PHYSICAL HARDWARE LAYER CONSTRAINTS

The Sovereign Shards framework is specifically tuned to run within the strict hardware boundaries of unprivileged consumer-grade workstations:

Resource Profile	Boundary Target	Architectural Rationale
System RAM	Max 16 GB	OS (4-6GB) + local llama.cpp server + memory layers must run concurrently without disk paging.
Execution Budget	4.5 GB Max	Hard limit for the active multi-agent memory space.
Context Window	2048 Tokens	Standard limit (redlines at 4096 on CPU-only chips). Forces aggressive context trimming and surgical BM25 memory lookups.
VRAM Bounds	0 MB (Intel HD 530)	No reliable GPU offloading. `GPU_DEVICE=none` must be declared; execution runs entirely on CPU threads.
Storage Bounds	USB 2.0 (FAT32)	Read speeds capped at ~30 MB/s. Models must be split beneath the 4 GB file boundary.

APPENDIX B: THE 20-TOOL CORE DEV CATALOG

Sovereign Shards maps external tools to Python scripts inside `tools/run/` . This catalog lists all registered operations available through the `ToolRegistry` :

1. `run_bash` : Executes shell commands within isolated subprocess environments. Automatically maps command payloads to standard input.
2. `run_read` : Reads local text assets using a chunked 40-line maximum limit with truncation alerts.
3. `run_write` : Writes string buffers directly to files. Supports both overwrite and append modes.
4. `run_search` : Leverages a fast regex grep-engine to scan directories for specific patterns.
5. `run_tree` : Outputs folder structures using customizable depth constraints and Unicode-safe characters.

tracking).

7. `run_test` : Runs the Python unit test framework (`unittest`) and reports pass/fail stats.
8. `run_lint` : Runs the static analysis linter (`ruff`) over specified directory paths.
9. `run_calc` : Executes safe mathematical calculations through an AST parser with no raw `eval()` usage.
10. `run_shield` : Hardens shard security (integrity checks, hash generation, malware scanning).
11. `run_scan` : Runs host port audits, credentials sweeps, and permission evaluations.
12. `run_bridge` : Compiles scanning files into Markdown guides and automated fix scripts.
13. `run_stats` : Outputs lines-of-code metrics, function counts, and codebase comments.
14. `run_scaffold` : Seeds project structures and directory assets for local workspaces.
15. `run_patch` : Inspects code files and applies unified diff updates to target targets.
16. `run_web_search` : Scrapes and parses search results from external sources (such as DuckDuckGo).
17. `run_github_tree` : Uses the GitHub Git Data API to read remote directory nodes recursively.
18. `run_github_read` : Pulls file contents from specified remote GitHub branches.
19. `run_github_write` : Commits single file changes via the remote contents API.
20. `run_github_pr` : Programmatically generates remote pull request requests.

APPENDIX C: ALTERNATIVE MODEL PROFILES

While Sovereign Shards is optimized for the local sub-3B matrix, it scales naturally to larger models depending on available hardware. This reference profiles alternative operational settings:

```
# ================================================================================
# PROFILE 1: ULTRA-LIGHTWEIGHT (Baseline Local Shard)
# Hardware: 16GB RAM Laptop (Intel CPU only)
# ================================================================================
LLAMA_MODEL_PATH=models/qwen2.5-coder-1.5b-instruct-q4_k_m.gguf
OLLAMA_NUM_CTX=2048
OLLAMA_NUM_PREDICT=256
OLLAMA_NUM_THREAD=4
GPU_DEVICE=none
GPU_LAYERS=0
J_TOOL_BUDGET=3

# ================================================================================
# PROFILE 2: HIGH-FIDELITY DEV (Standard Shard)
# Hardware: 16GB RAM + Integrated Iris/M1-Class GPU
# ================================================================================
LLAMA_MODEL_PATH=models/qwen2.5-coder-7b-instruct-q4_k_m.gguf
OLLAMA_NUM_CTX=4096
OLLAMA_NUM_PREDICT=512
OLLAMA_NUM_THREAD=6
GPU_DEVICE=vulkan
GPU_LAYERS=20
```

```
# ================================================================================
# PROFILE 3: ENTERPRISE CLUSTERS (Cloud / On-Premise)
# Hardware: Nvidia A10G / Dedicated Server Nodes
# ================================================================================
RUNTIME_BACKEND=ollama
OLLAMA_NUM_CTX=16384
OLLAMA_NUM_PREDICT=1024
OLLAMA_NUM_THREAD=12
J_TOOL_BUDGET=10
```